

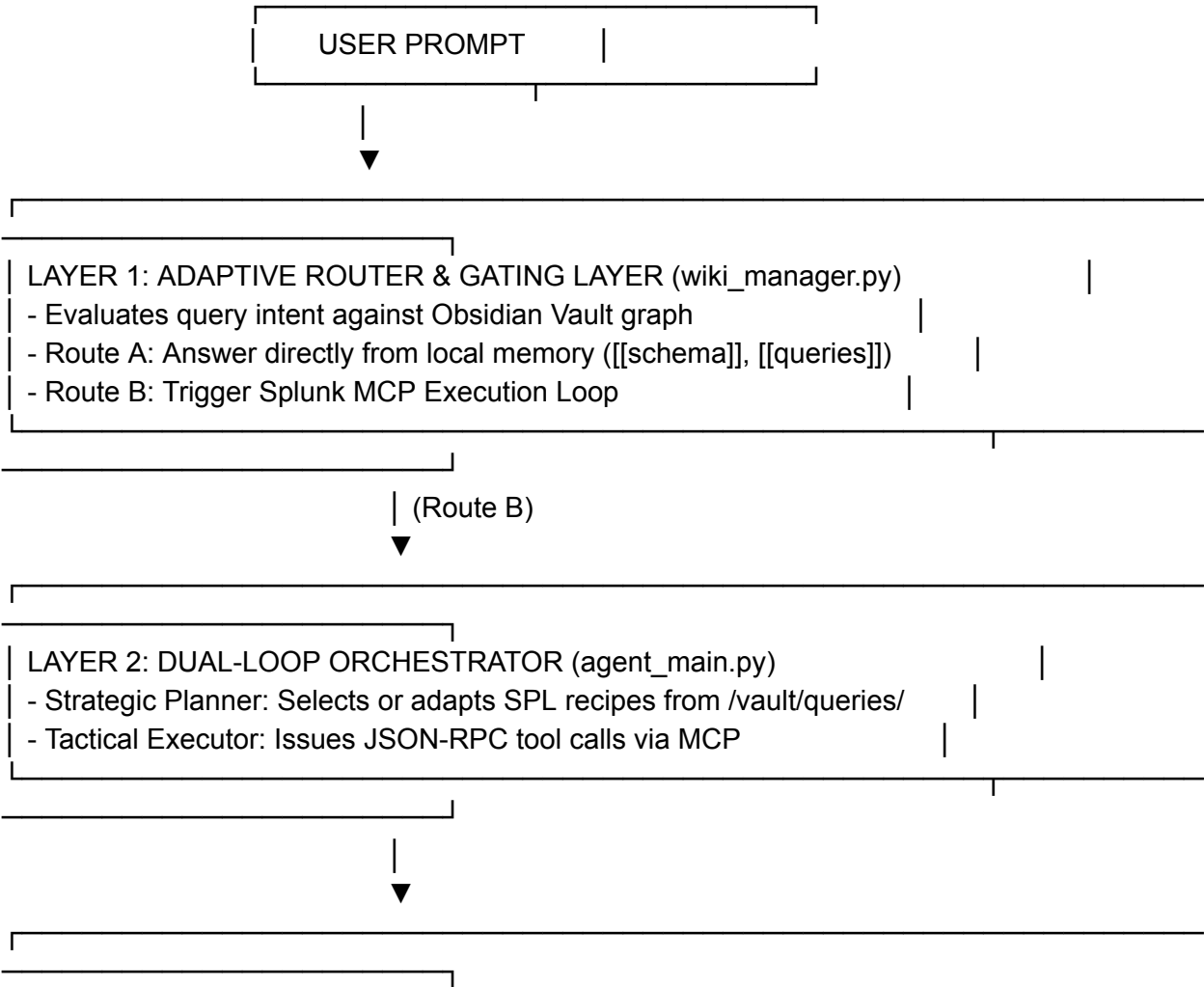
ARCHITECTURE SPECIFICATION: DUAL-LOOP HYBRID-MCP MEMORY-COMPILER AGENT

OVERVIEW

You are an expert Principal AI Software Engineer. Your task is to generate a production-ready, fully type-annotated Python implementation for an **Autonomous Splunk-MCP Agent**.

This system connects a local **Ollama model** (`llama3.1:8b`` or `hermes3:8b``) to a **Splunk MCP Server** using a **Dual-Loop Memory-Compiler Pattern**. It relies on an **Obsidian Vault (LLM-Wiki)** as persistent knowledge-graph memory and a **Map-Reduce Payload Chunker** for context window safety.

DESIGN PATTERN ARCHITECTURE



LAYER 3: MAP-REDUCE LOG COMPILER & CONTEXT PRUNER (log_chunker.py)

- Strips high-cardinality metadata headers (keeps status, host, message)
- Summarizes 10,000+ token log payloads into 200-token Markdown tables
- Purges raw JSON payloads from active prompt context immediately

- ↓

LAYER 4: PROCEDURAL GRAPH PERSISTENCE (wiki_manager.py) - Appends synthesized summary note to /vault/incidents/ - Auto-saves validated SPL patterns to /vault/queries/ - Replaces active memory state with pointer link: [[incidents/incident_id]]	
---	--

-

MODULE SPECIFICATIONS TO IMPLEMENT

1. `wiki\_manager.py` (Obsidian LLM-Wiki Layer)

*****Directory_INITIALIZER:**** Automatically create ``/vault/index.md``, ``/vault/schema/indexes.md``, ``/vault/schema/sourcetypes.md``, ``/vault/queries/``, and ``/vault/incidents/``.

```
* **`search_graph(query: str) -> str`:** Parse `index.md` and related Markdown files using regex for `[[wikilinks]]`. Extract matching context blocks (max 500 tokens).
```

```

**`save_spl_recipe(name: str, spl: str, description: str, tags: list[str]):** Store validated SPL
snippets as structured Markdown notes in `/vault/queries/` with YAML frontmatter.

```

```
* **`persist_incident(title: str, summary: str, source_query: str) -> str`** Create a new
timestamped Markdown note in `/vault/incidents/` containing the compiled findings and
back-links to related schemas. Return the `[[wikilink]]` pointer string.
```

2. `mcp\_client.py` (Splunk MCP Interface)

* Wrap `mcp.ClientSession` to connect to `splunk-mcp-server` over stdio/JSON-RPC.

- * Implement safety-wrapped async methods:

```
* `splunk get metadata`
```

```
* `splunk run query(spl query: str, time range: str = "-15m", limit: int = 100)`
```

****Guardrails:**** Automatically validate query strings to ensure index parameters exist and enforce result limits (`count=100`).

3. `log\_chunker.py` (Payload Compiler & Context Pruner)

* **`LogChunker` Class:**

* **Token Estimation:** Calculate approximate token size of Splunk JSON output.

* **Field Stripping:** If payload > 4,000 tokens, strip meta-headers, retaining only `\_time`, `host`, `sourcetype`, `status`, and `\_raw` error text.

* **Map-Reduce Summarizer:** Slice remaining logs into 1,000-token chunks. Execute a fast local Ollama summarization pass over each slice, then reduce slice summaries into a single Markdown table.

* **Context Purging:** Immediately remove raw JSON log blocks from conversation memory after compilation.

4. `agent\_main.py` (Dual-Loop Orchestrator & State Machine)

* Initialize Ollama client (`llama3.1:8b` or `hermes3:8b`) with `num\_ctx=32768` and `temperature=0.1`.

* Implement the agent control loop:

1. Inspect local Obsidian Wiki graph first (Route A vs. Route B gating).
2. If Splunk data is needed, select an SPL template from `/vault/queries/` or construct a guarded SPL query.
3. Execute tool call via `mcp\_client.py`.
4. Pass raw payload to `log\_chunker.py` for Map-Reduce compression if token size exceeds threshold.
5. Save findings to `/vault/incidents/` via `wiki\_manager.py` and output final answer with link pointers.

OLLAMA SYSTEM PROMPT FOR 8B MODEL

```text

You are an Autonomous Splunk Investigator paired with an Obsidian LLM-Wiki memory graph.

### RULES:

1. ALWAYS inspect local Wiki notes before constructing a Splunk search.
2. When formulating SPL, copy or adapt proven templates from /vault/queries/ to prevent syntax errors.
3. Always specify an explicit index and tight time range (-15m, -1h) in splunk\_run\_query calls.
4. NEVER output raw log dumps into final answers. Summarize key findings into clean Markdown tables.

5. Save any new working SPL patterns back to /vault/queries/ for future reuse.  
...

---

## ## INSTRUCTIONS FOR COPILOT / FRONTIER MODEL

Generate complete, clean, type-annotated Python code implementing `wiki\_manager.py`, `mcp\_client.py`, `log\_chunker.py`, and `agent\_main.py` following these precise architectural specifications. Include async handling, exception handling for MCP JSON-RPC timeouts, and Ollama context safety checks.